Создание страниц регистрации и авторизации

В предыдущей лекции для таблицы с пользователями мы создали CRUD модель для отображения. Помимо сгенерировали форму для создания нового пользователя, следовательно, эту форму можно использовать для регистрации новых пользователей.

Добавим в меню навигации ссылку на форму для создания нового пользователя. Меню навигации находится в каталоге view, подкаталог layouts, файл main.php.

Структура кода представляет собой вывод навигационного меню (виджета Nav) на веб-сайте.

options – массив настроек HTML-элементов контейнера виджета.

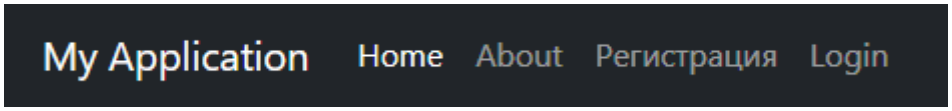
items – массив пунктов меню, каждый элемент которого представляет собой отдельную ссылку или группу ссылок.

```
echo Nav::widget([
    'options' => ['class' => 'navbar-nav'],
    'items' => [
        ['label' => 'Home', 'url' => ['/site/index']],
        ['label' => 'About', 'url' => ['/site/about']],
        ['label' => 'Contact', 'url' => ['/site/contact']],
        Yii::$app->user->isGuest
            ? ['label' => 'Login', 'url' => ['/site/login']]
            : '<li class="nav-item">'
                . Html::beginForm(['/site/logout'])
                . Html::submitButton(
                    'Logout (' . Yii::$app->user->identity->username . ')',
                    ['class' => 'nav-link btn btn-link logout']
                )
                . Html::endForm()
                . '</li>'
    ]
]);
```

Поменяем ссылку Contact на ссылку для регистрации. Название ссылки – Регистрация, путь /user/create.

```
['label' => 'Регистрация', 'url' => ['/user/create']],
```

Отображение меню навигации выглядит теперь следующим образом.



Переходим на страницу регистрации. Нас встречает форма для создания нового пользователя.

В файле User.php подкаталога models к расширению \yii\db\ActiveRecord прописать интерфейс для реализации механизма идентификации пользователей в фреймворке Yii2.

Active Record — это паттерн проектирования, широко применяемый в ORM (Object Relational Mapping). Его суть заключается в представлении каждой строки таблицы базы данных в виде отдельного экземпляра класса, что упрощает работу с базой данных, делая её более декларативной и удобной.

```
class User extends \yii\db\ActiveRecord
```

```
class User extends \yii\db\ActiveRecord implements \yii\web\IdentityInterface
```

Их ранее скопированного содержимого файла User.php скопируем функции этого интерфейса. Видоизменим некоторые эти функции.

Метод findIdentity() — это важный компонент системы аутентификации в Yii2, позволяющий находить пользователя по уникальному идентификатору (id). Он реализует одноимённый метод интерфейса \yii\web\IdentityInterface, который обязателен для моделей, используемых в процессе авторизации.

```
public static function findIdentity($id)
{
    return isset(self::$users[$id]) ? new static(self::$users[$id]) : null;
}
```

Вместо содержимого используем свой – self::findOne($id): Это стандартный способ обращения к классу внутри самого себя (через ключевое слово self). Функция findOne() ищет первую запись в базе данных,

соответствующую переданным условиям. В данном примере условие одно — найти запись с указанным полем id.

```
public static function findIdentity($id)
{
    return self::findOne($id)
}
```

Метод findIdentityByAccessToken() является важным элементом системы аутентификации в Yii2, позволяющим искать пользователя по специальному токену доступа. Этот метод предусмотрен интерфейсом \yii\web\IdentityInterface, который обязательно должен быть реализован в моделях, использующих систему авторизации.

Очищаем содержимое этой функции.

```
public static function findIdentityByAccessToken($token, $type = null)
{

}
```

Метод findByUsername() — это удобный способ поиска пользователя по его имени пользователя (логину). Такой метод часто добавляется в классы моделей, работающие с пользователями, чтобы упростить процедуру аутентификации и загрузки данных пользователя.

```
public static function findByUsername($username)
{
    return self::find()->where(['login_user' => $username])->one();
}
```

Метод getId() в Yii2 является важной частью системы аутентификации и авторизации. Он входит в интерфейс \yii\web\IdentityInterface, который обязаны реализовать модели, используемые для идентификации пользователей.

Метод возвращает уникальный идентификатор текущего пользователя. Этот идентификатор (чаще всего — числовой id из базы данных) служит для однозначной идентификации пользователя в рамках приложения.

```
public function getId()
{
    return $this->id_user;
}
```

Метод getAuthKey() является обязательной частью реализации интерфейса \yii\web\IdentityInterface в Yii2 и отвечает за возврат уникального ключа авторизации пользователя. Этот ключ используется для подтверждения подлинности пользователя в ходе сессии или повторного входа без повторной отправки пароля.

Очищаем содержимое этой функции.

```
public function getAuthKey()
{

}
```

Метод validateAuthKey() в Yii2 является частью реализации интерфейса \yii\web\IdentityInterface и предназначен для проверки подлинности ключа авторизации пользователя. Он отвечает за сравнение переданного ключа авторизации с тем, который хранится в базе данных или модели пользователя.

Очищаем содержимое этой функции.

```
public function validateAuthKey($authKey)
{

}
```

Метод validatePassword() — это критически важный элемент системы аутентификации в Yii2, используемый для проверки введённого пользователем пароля против зашифрованного пароля, хранящегося в базе данных.

```
public function validatePassword($password)
{
    return $this->password_user === $password;
}
```

Скопируем файл User.php и переименуем его в RegForm.php. Этот файл осуществляет создание моделей для регистрации.

RegForm.php	01.01.2026 14:01	Исходный файл ...	3 КБ

Наша модель расширяет класс модели с пользователями.

```
class RegForm extends User
```

Удаляем все содержимое этого класса за исключением функций: tableName(), rules() и attributeLabels().

```php
class RegForm extends User
{


    /**
     * {@inheritdoc}
     */
    public static function tableName()
    {
        return 'user';
    }

    /**
     * {@inheritdoc}
     */
    public function rules()
    {
        return [
            [['admin_user'], 'default', 'value' => 0],
            [['fio_user', 'login_user', 'email_user', 'password_user'], 'required'],
            [['admin_user'], 'integer'],
            [['fio_user', 'login_user', 'email_user', 'password_user'], 'string', 'max' => 255],
            [['login_user'], 'unique'],
        ];
    }

    /**
     * {@inheritdoc}
     */
    public function attributeLabels()
    {
        return [
            'id_user' => 'Id User',
            'fio_user' => 'Fio User',
            'login_user' => 'Login User',
            'email_user' => 'Email User',
            'password_user' => 'Password User',
            'admin_user' => 'Admin User',
        ];
    }
}
```

Добавим два атрибута для реализации полей формы регистрации: повторение пароля и согласия на обработку данных.

```php
public $passwordConfirm;
public $agree;
```

Пропишем эти атрибуты в массив в функции attributeLabels(), а также русифицируем названия для полей.

```php
public function attributeLabels()
{
    return [
        'id_user' => 'Id User',
        'fio_user' => 'ФИО',
        'login_user' => 'Login',
        'email_user' => 'Email',
        'password_user' => 'Пароль',
        'admin_user' => 'Admin',
        'passwordConfirm' => 'Admin',
        'agree' => 'Даю согласие на обратку персональных данных'
    ];
}
```

Открываем файл UserController.php подкаталога controllers. В функции создания пользователя actionCreate().

Создаем новый экземпляр модели формы регистрации (RegForm).

Далее если пришел POST-запрос (форма была отправлена), модель загружает переданные данные и пытается сохранить их:

if ($this->request->isPost) { if ($model->load($this->request->post()) && $model->save()) {}}

После успешного сохранения данных, система производит автоматический вход зарегистрированного пользователя и перенаправляет его на страницу просмотра.

Если запрос GET (страница открытия формы регистрации), модель инициализирует значения по умолчанию (если предусмотрены правилами валидации):

else {$model->loadDefaultValues();}

Независимо от способа перехода (GET или POST), страница создается методом рендера:

return $this->render('create', [

'model' => $model,

]);

```php
public function actionCreate()
{
    $model = new RegForm();

    if ($this->request->isPost) {
        if ($model->load($this->request->post()) && $model->save()) {
            return $this->redirect(['view', 'id_user' => $model->id_user]);
        }
    } else {
        $model->loadDefaultValues();
    }

    return $this->render('create', [
        'model' => $model,
    ]);
}
```

Чтобы модель RegForm выполняла действия необходимо вызвать ее.

```php
use app\models\User;
use app\models\RegForm;
use app\models\UserSearch;
use yii\web\Controller;
use yii\web\NotFoundHttpException;
use yii\filters\VerbFilter;
```

На данный момент наша форма регистрации выглядит следующим образом.

Необходимо добавить следующие поля: подтверждение пароля и чекбокс для согласия на обработку данных.

Откроем файл _form.php подкаталога user, каталога views. Создадим еще одно текстовое поле с помощью $form – переменной, содержащей объект формы, метода field(), который создаёт отдельный элемент формы для указанного атрибута модели, и метода textInput(), который выводит простое текстовое поле.,

а атрибут maxlength ограничивает максимальное количество символов, которое можно ввести в данное поле.

```php
<?php

use yii\helpers\Html;
use yii\widgets\ActiveForm;

/** @var yii\web\View $this */
/** @var app\models\User $model */
/** @var yii\widgets\ActiveForm $form */
?>

<div class="user-form">

    <?php $form = ActiveForm::begin(); ?>

    <?= $form->field($model, 'fio_user')->textInput(['maxlength' => true]) ?>

    <?= $form->field($model, 'login_user')->textInput(['maxlength' => true]) ?>

    <?= $form->field($model, 'email_user')->textInput(['maxlength' => true]) ?>

    <?= $form->field($model, 'password_user')->textInput(['maxlength' => true]) ?>

    <?= $form->field($model, 'passwordConfirm')->textInput(['maxlength' => true]) ?>

    <?= $form->field($model, 'agree')->checkbox() ?>

    <div class="form-group">
        <?= Html::submitButton('Save', ['class' => 'btn btn-success']) ?>
    </div>

    <?php ActiveForm::end(); ?>

</div>
```

В конечном итоге интерфейс регистрационной формы выглядит следующим образом.

Метод rules() в Yii2 предназначен для задания правил валидации атрибутов модели. Эти правила определяют, каким требованиям должны соответствовать данные, переданные пользователем через форму или иным способом.

Чтобы прописать правила валидации в функции rules необходимо открыть файл RegForm.php.

```
public function rules()
{
```

Первое правило – обязательные поля для заполнения.

```
[['fio_user', 'login_user', 'email_user', 'password_user', 'passwordConfirm', 'agree'], 'required', 'message' => 'Поле обязательно для заполнение'],
```

Массив с полями, для которых будет использоваться правило.

```
['fio_user', 'login_user', 'email_user', 'password_user', 'passwordConfirm', 'agree'],
```

Следующий атрибут – само правило, required.

```
'required',
```

Третий атрибут – сообщение, которое отобразится если правило не будет соблюдено.

```
'message' => 'Поле обязательно для заполнение'],
```

Таким образом, если поле оставить незаполненным, то мы увидим сообщение-предупреждение.

Второе правило – сравнение символов в ФИО пользователя с рекомендуемыми символами.

```
['fio_user', 'match', 'pattern' => '/^[А-Яа-я\s\-]{5,}$/u', 'message' => 'Только кириллица, пробелы и дефис'],
```

Параметр match используется для проверки соответствия строки определённому регулярному выражению pattern.

Сама строка начинается ^ и заканчивается $. [а-яА-Я] – разрешается использование русскоязычных символов. [\s] – разрешается использование пробелов. [\-] – разрешается использование дефисов. {5,} –минимальное и максимальное количество элементов.

```
'match', 'pattern' => '/^[А-Яа-я\s\-]{5,}$/u',
```

Таким образом, если поле заполнить неправильно, то мы увидим сообщение-предупреждение.

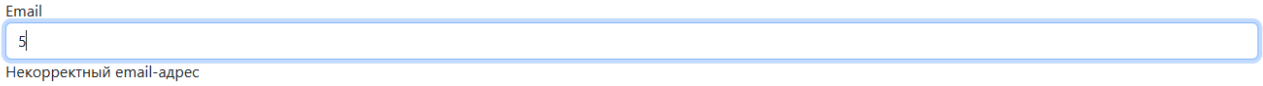
Аналогично проделайте шаг для логина, допустимы латинские символы и цифры.

```
['login_user', 'match', 'pattern' => '/^[a-zA-Z0-9]{1,}$/u', 'message' => 'Только латинские буквы и цифры'],
```

Правило для проверки электронной почты – просто параметр email.

```
['email_user', 'email', 'message' => 'Некорректный email-адрес'],
```

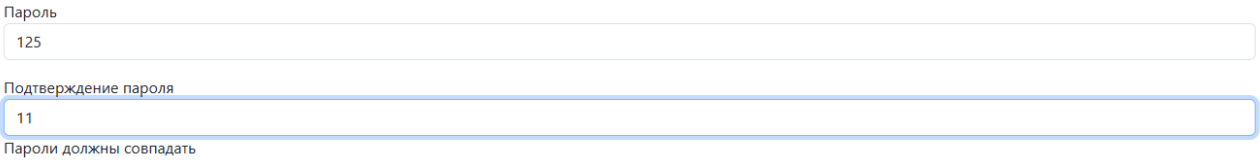
Таким образом, если поле заполнить неправильно, то мы увидим сообщение-предупреждение.



Параметр compare – обеспечивает совпадение поля подтверждения пароля с основным полем пароля.

```
['passwordConfirm', 'compare', 'compareAttribute' => 'password_user', 'message' => 'Пароли должны совпадать'],
```

Таким образом, если поля не совпадают, то мы увидим сообщение-предупреждение.



Первое правило для чекбокса – логический тип данный.

Второе – сравнение с логическим значение – true.

```
['agree', 'boolean'],
['agree', 'compare',  'compareValue' => true, 'message' => 'Необходимо согласиться'],
```

Таким образом, если чекбокс не отмечен, то мы увидим сообщение-предупреждение.

☐ Даю согласие на обратку персональных данных

Необходимо согласиться

В UserController в функции actionCreate упростим следующий код.

```
if ($this->request->isPost) {
    if ($model->load($this->request->post()) && $model->save()) {
        return $this->redirect(['/user', 'id_user' => $model->id_user]);
    }
```

```
if ($this->request->isPost) {
    if ($model->load($this->request->post()) && $model->save()) {
        return $this->redirect(['/user']);
    }
```

Теперь при регистрации пользователя, он автоматически заносится в базу данных.

Create User

ФИО

Бета Пользователь

Login

BetaTester

Email

BetaTester@gmail.com

Пароль

123456789

Подтверждение пароля

123456789

☑ Даю согласие на обратку персональных данных

Users

Create User

Showing **1-1** of **1** item.

#	Id User	Fio User	Login User	Email User	Password User	
1	1	Бета Пользователь	BetaTester	BetaTester@gmail.com	123456789	

Добавим автоматический вход после регистрации пользователя.

Yii::$app->user->login($model);

```
if ($this->request->isPost) {
    if ($model->load($this->request->post()) && $model->save()) {
        Yii::$app->user->login($model);
        return $this->redirect(['/user']);
    }
```

Чтобы отобразить в меню навигации, что авторизация прошла. Пропишем в файле main.php подкаталога layouts каталога views.

```
'Logout (' . Yii::$app->user->identity->login_user . ')',
```

Logout (Ivanov777)